Testování aplikace pro objednávání zmrzliny

Michal Hrouda

v1.0.0
22. 4. 2026

Obsah

Zadání webové aplikace . . . 2

Popis funkcionality aplikace . . . 2

Testovací strategie . . . 5
- Uživatelská role . . . 5
  - Testovací účty . . . 5
- Zákaznická část . . . 6
- Administrátorská část . . . 6
- Prioritizace částí aplikace . . . 6

Test levels . . . 7

Testy vstupů . . . 7
- Konfigurátor zmrzliny . . . 7
  - Přehled vstupů formuláře . . . 7
  - Třídy ekvivalence . . . 8
  - Pairwise testování konfigurátoru . . . 10
    - Faktory a hodnoty . . . 10
    - Pairwise matice — 9 testovacích případů . . . 10

Testy průchodů . . . 11
- Popis procesu . . . 11
- CPT manager diagram . . . 12

Implementace testů . . . 13
- IceCreamConfigurationPairwiseTests.java . . . 13
- CreateOrderProcessTests.java . . . 13
- NoAccountClientOrderTests.java . . . 13
- AuthClientOrderTests.java . . . 14
- AdminPageTests.java . . . 14

Zadání webové aplikace

Testovaná webová aplikace byla vygenerována pomocí nástroje **Claude Opus 4**. Zadání znělo: vytvořit e-commerce webovou aplikaci postavenou na frameworku **Spring Boot**, která umožní zákazníkům online objednávání zmrzliny z více zmrzlináren.

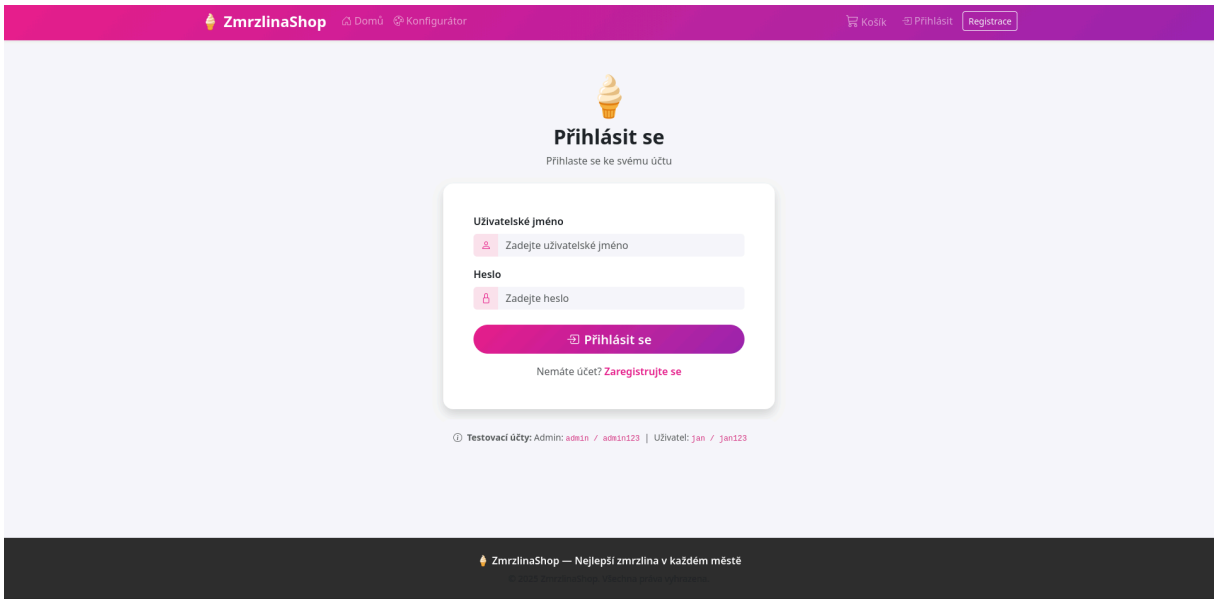
Aplikace měla splňovat následující požadavky:

- výběr pobočky (zmrzlinárny) a přidání produktů do košíku
- konfigurátor vlastní zmrzliny (výběr základu, kopečků, toppingů a sušenek)
- možnost **registrace a přihlášení** zákazníka
- přehled vlastních objednávek pro přihlášeného uživatele
- **administrátorská sekce** pro správu objednávek, produktů a obchodů
- sledování stavu zásob produktů

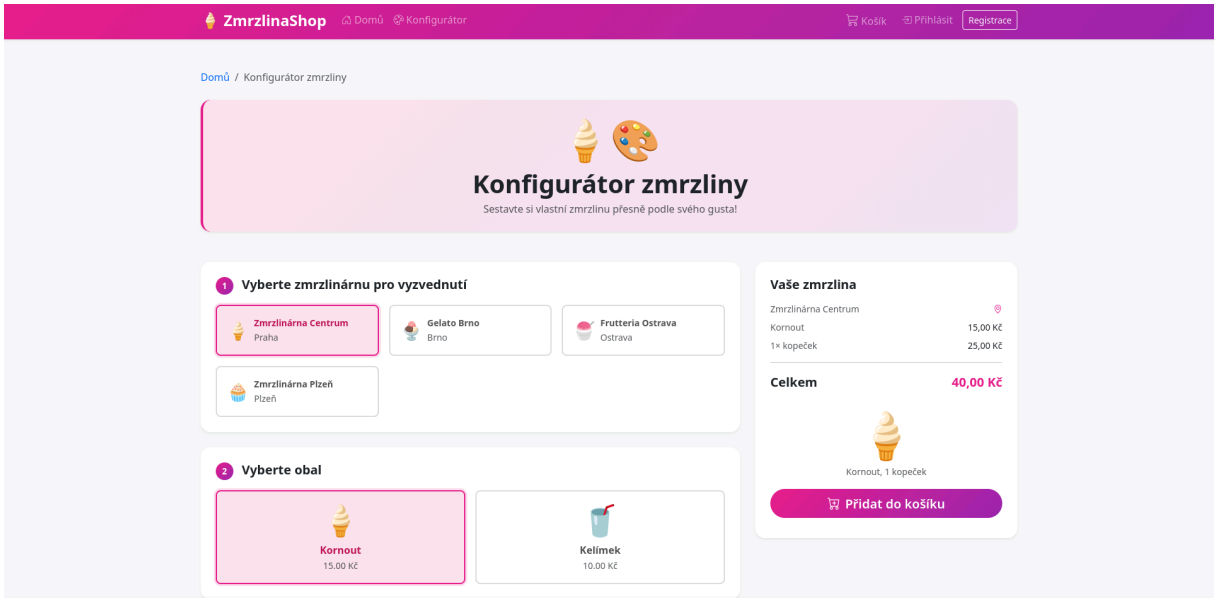
Popis funkcionality aplikace

Webová e-commerce aplikace pro objednávání zmrzliny online. Zákazník si vybere pobočku, přidá produkty do košíku nebo si sestaví vlastní zmrzlinu pomocí konfigurátoru, a odešle objednávku k vyzvednutí.

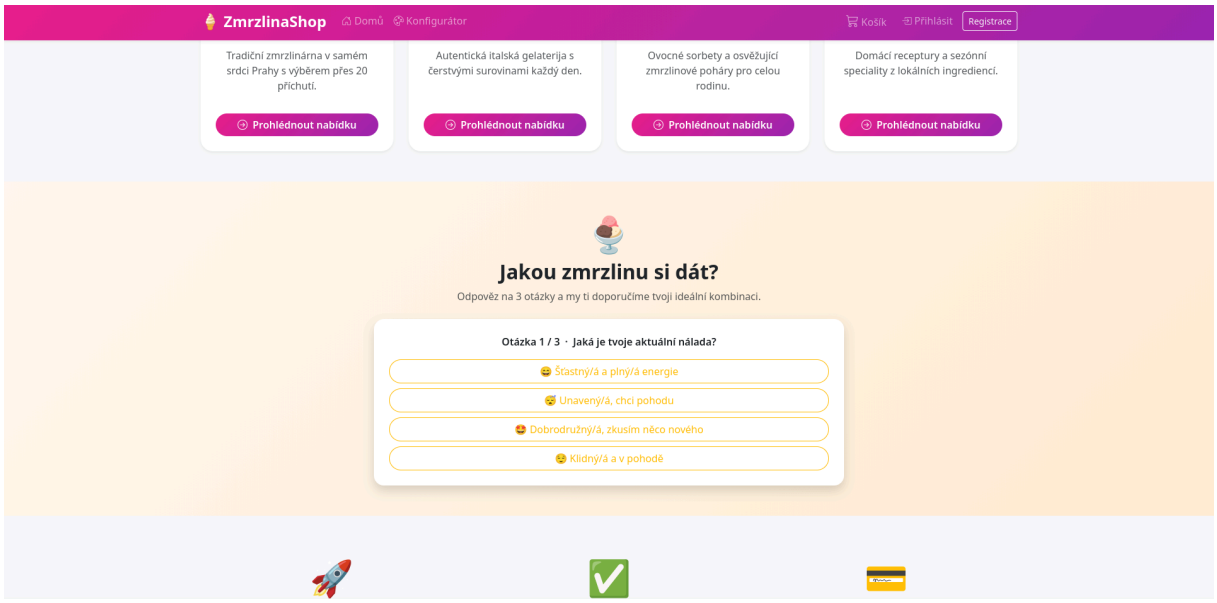
Obrázek 1: Screenshot hlavní stránky



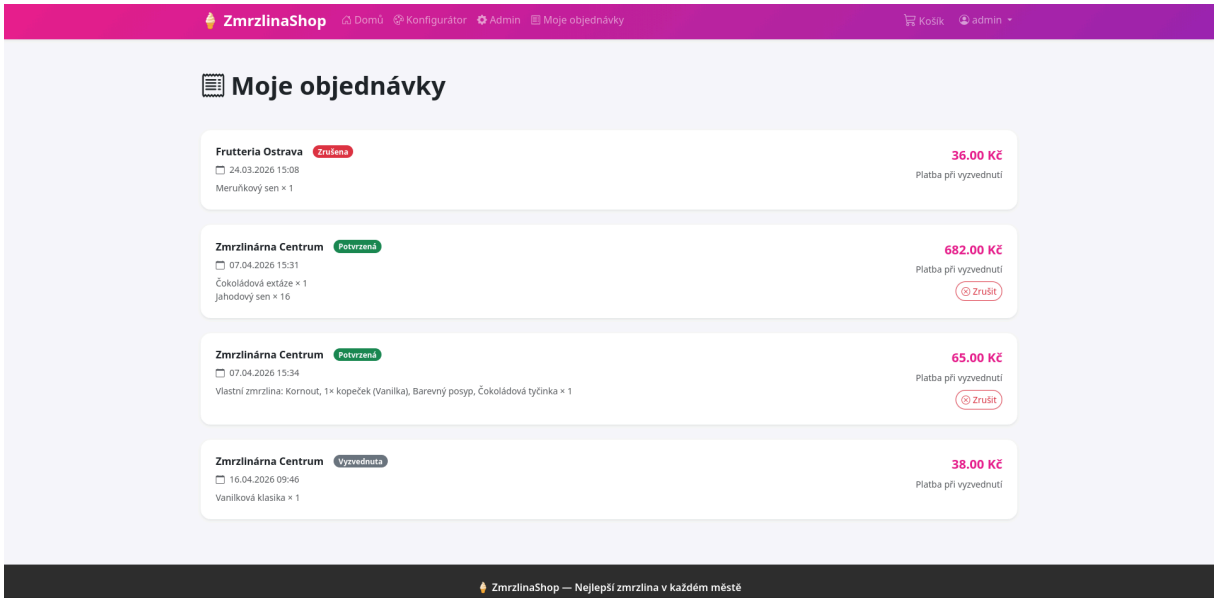
Obrázek 2: Screenshot login stránky



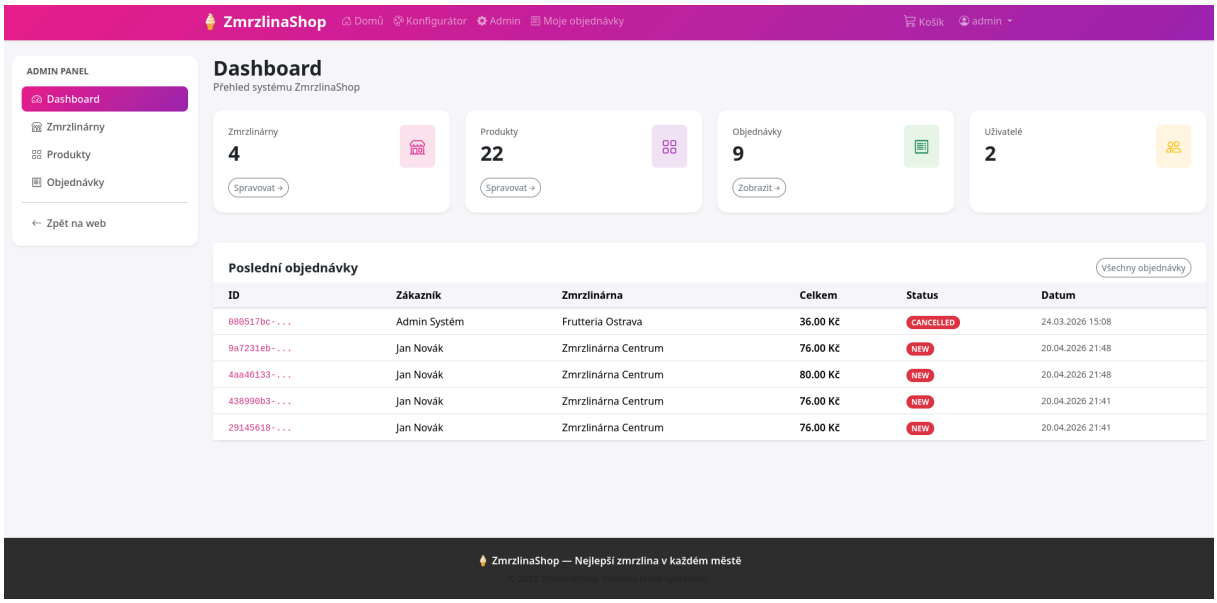
Obrázek 3: Screenshot stránky konfigurátoru zmrzliny



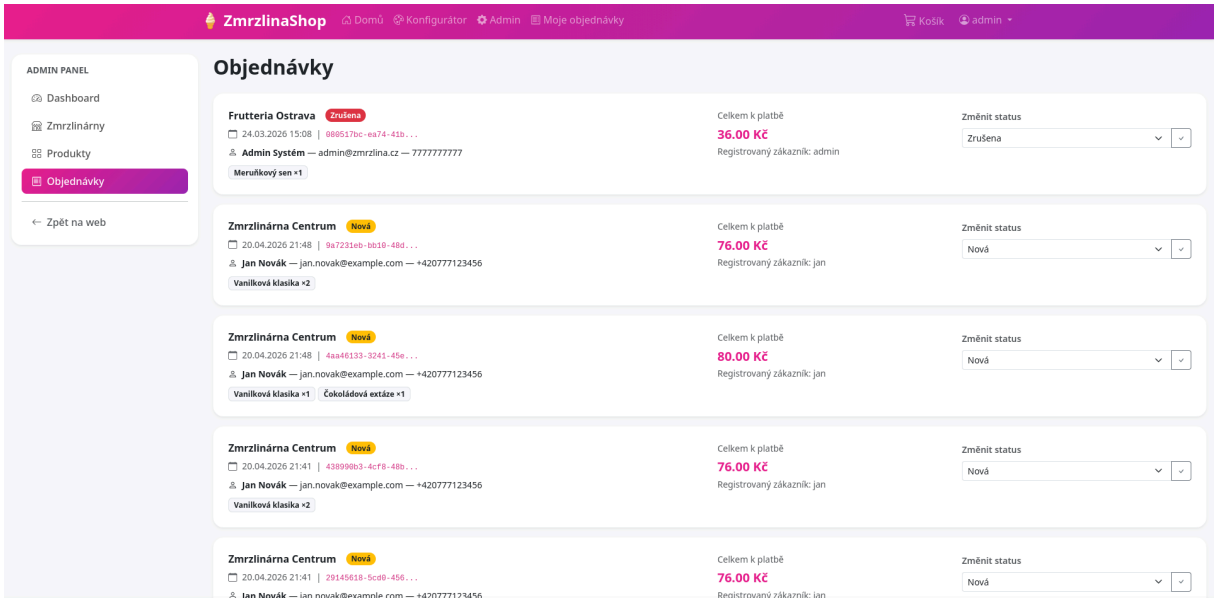
Obrázek 4: Screenshot kvízu pro výběr zmrzliny



Obrázek 5: Screenshot stránky s objednávkami uživatele



Obrázek 6: Screenshot admin dashboardu



Obrázek 7: Screenshot admin stránky pro správu objednávek

Testovací strategie

Uživatelská role

Uživatel může nabývat tří rolý **nepřihlášený uživatel, přihlášený uživatel a admin**, kde každá role přidává uživateli nové funkcionality

- Nepřihlášený uživatel = možnost objednání zmrzliny, konfigurátor zmrzliny, filtrace nabýdky a kvíz
- Přihlášený uživatel = uchovávání objednávek, předvyplnění informací u objednávání
- Admin = dashboard, správá produktů a kamenných prodejen

Testovací účty

- Admin

Uživatelské jméno: **admin**
Heslo: **heslo123**

- Uživatel
  Uživatelské jméno: **jan**
  Heslo: **jan123**

Zákaznická část

Stránka	URL adresa
Hlavní stránka	`/`
Detail zmrzlinárny	`/shop/{id}`
Košík	`/cart`
Konfigurátor zmrzliny	`/configurator`
Pokladna	`/order/checkout`
Potvrzení objednávky	`/order/confirmation`
Moje objednávky	`/order/my-orders`
Registrace	`/register`
Login	`/login`

Administrátorská část

Stránka	URL adresa
Dashboard	`/admin`
Správa zmrzlináren	`/admin/shops`
Správa produktů	`/admin/products`
Správa objednávek	`/admin/orders`

Prioritizace částí aplikace

1) Zákaznická část aplikace 1) Část aplikace pro přihlášené zákazníky 2) Admin část aplikace 3) Kvíz pro výběr zmrzliny

Test levels

		Pravděpodobnost selhání		
		High	Medium	Low
Možné poškození v případě selhání	High	A	B	B
	Medium	B	B	C
	Low	C	C	C

Třída rizika

Obrázek 8: Tabulka definice priorit testů

Část aplikace	Třída rizika	Test	Důvod
Objednávkový proces (košík + pokladna)	A	**CreateOrderProcessTests.java**	Klíčový proces aplikace s množstvím větvení; selhání znamená ztrátu objednávky.
Konfigurátor zmrzliny	B	**IceCreamConfigurationPairwiseTests.java**	Vysoký počet kombinací vstupů; pairwise pokrývá rizika za přijatelný počet testů.
Zákaznický flow (nepřihlášený)	A	**NoAccountClientOrderTests.java**	Většinová skupina uživatelů; přímý dopad na konverzi a tržby.
Zákaznický flow (přihlášený)	B	**AuthClientOrderTests.java**	Menší skupina uživatelů, ale pokrývá historii objednávek a jejich rušení.
Administrátorská sekce	B	**AdminPageTests.java**	Nižší četnost použití, avšak chyba může ovlivnit data celé aplikace.
Kvíz pro výběr zmrzliny	C	**NoAccountClientOrderTests.java**	Pomocná funkcionalita bez perzistence dat; nízký dopad na uživatele.
Autentizace a autorizace	B	**SeleniumBaseTest.java**	Předpoklad ostatních testů; selhání se projeví napříč celou testovací sadou.

Testy vstupů

Konfigurátor zmrzliny

Jako testovaný formulář byl zvolen **konfigurátor vlastní zmrzliny** (`/configurator`). Jedná se o pětikrokový formulář, který umožňuje zákazníkovi sestavit zmrzlinu výběrem zmrzlinárny, obalu, počtu a příchutí kopečků, posypek a sušenek. Výsledná cena je počítána na serveru. Formulář je netriviální — kombinuje povinné i volitelné vstupy, číselná omezení i omezení výčtem hodnot.

Přehled vstupů formuláře

Pole	Typ	Povinné	Omezení
`shopId`	radio	ano	musí existovat v systému
`kornout/kelímek`	radio	ano	`"kornout"` nebo `"kelimek"`
`flavors`	select (1–5×)	ano	min 1, max 5; hodnota z předdefinovaného seznamu 15 příchutí
`selectedToppings`	checkboxy	ne	0–6; hodnoty z předdefinovaného seznamu

Pole	Typ	Povinné	Omezení
`selectedCookies`	checkboxy	ne	0–5; hodnoty z předdefinovaného seznamu

Třídy ekvivalence

Zelené řádky = platná třída, červené = neplatná třída.

ID	Pole	Třída	Popis	Výsledek
E1	`shopId`	platná	existující ID aktivní zmrzlinárny (např. `"shop-1"`)	OK — formulář se zpracuje
E2	`shopId`	neplatná	neexistující nebo prázdné ID	chyba: „Vybraná zmrzlinárna neexistuje.„
E3	`kornout/kelímek`	platná	`"kornout"` — cena 15 Kč	OK — použije se cena kornoutu
E4	`kornout/kelímek`	platná	`"kelimek"` — cena 10 Kč	OK — použije se cena kelímku
E5	`kornout/kelímek`	neplatná	jiná hodnota (mimo výčet)	tiše použije cenu kelímku (`isCone = false`)
E6	`flavors`	platná	1 kopeček — hraniční minimum	OK — cena = základ + 25 Kč
E7	`flavors`	platná	2–4 kopečky — střední rozsah	OK — cena odpovídá počtu
E8	`flavors`	platná	5 kopečků — hraniční maximum	OK — cena = základ + 125 Kč
E9	`flavors`	neplatná	0 kopečků (pole zcela chybí)	HTTP 400 — povinný parametr
E10	`flavors`	neplatná	6+ kopečků (obejití UI limitu)	server akceptuje — chybí max. validace
E11	`flavors`	neplatná	příchuť mimo předdefinovaný seznam	server akceptuje — chybí validace výčtu
E12	`toppings` / `cookies`	platná	žádný výběr (pole vynecháno)	OK — bez příplatku
E13	`toppings` / `cookies`	platná	1 až max−1 položek z výčtu	OK — cena navýšena

ID	Pole	Třída	Popis	Výsledek
E14	`toppings` / `cookies`	platná	přesně max položek (toppings: 6, cookies: 5) — hraniční maximum	OK — cena navýšena
E15	`toppings` / `cookies`	neplatná	počet položek nad maximum (obejití UI limitu)	server akceptuje — chybí max. validace
E16	`toppings` / `cookies`	neplatná	hodnota mimo předdefinovaný seznam	server akceptuje — chybí validace výčtu
E17	stav košíku	platná	košík je prázdný	OK — položka přidána
E18	stav košíku	platná	košík obsahuje produkty ze **stejné** zmrzlinárny	OK — položka přidána
E19	stav košíku	neplatná	košík obsahuje produkty z **jiné** zmrzlinárny	chyba: „Košík obsahuje produkty z jiné zmrzlinárny.„

Pairwise testování konfigurátoru

Technika **pairwise testing** (all-pairs) redukuje počet kombinací tak, aby každá dvojice hodnot libovolných dvou vstupních parametrů byla pokryta alespoň jedním testovacím případem. Namísto úplného kartézského součinu (jenž by u 4 faktorů s hodnotami $2 \times 3 \times 3 \times 3 = 54$ kombinací) stačí 9 testů.

Faktory a hodnoty

Faktor	Hodnoty	Poznámka
A — `kornout/kelímek`	`"kornout"`, `"kelimek"`	2 hodnoty — výčtový typ
B — kopečky	1, 3, 5	min, střed, max povoleného rozsahu
C — posypky	0, 3, 6	žádná, střed, max (6)
D — sušenky	0, 2, 5	žádná, střed, max (5)

Pairwise matice — 9 testovacích případů

Každý pár hodnot libovolných dvou faktorů se v tabulce vyskytuje alespoň jednou.

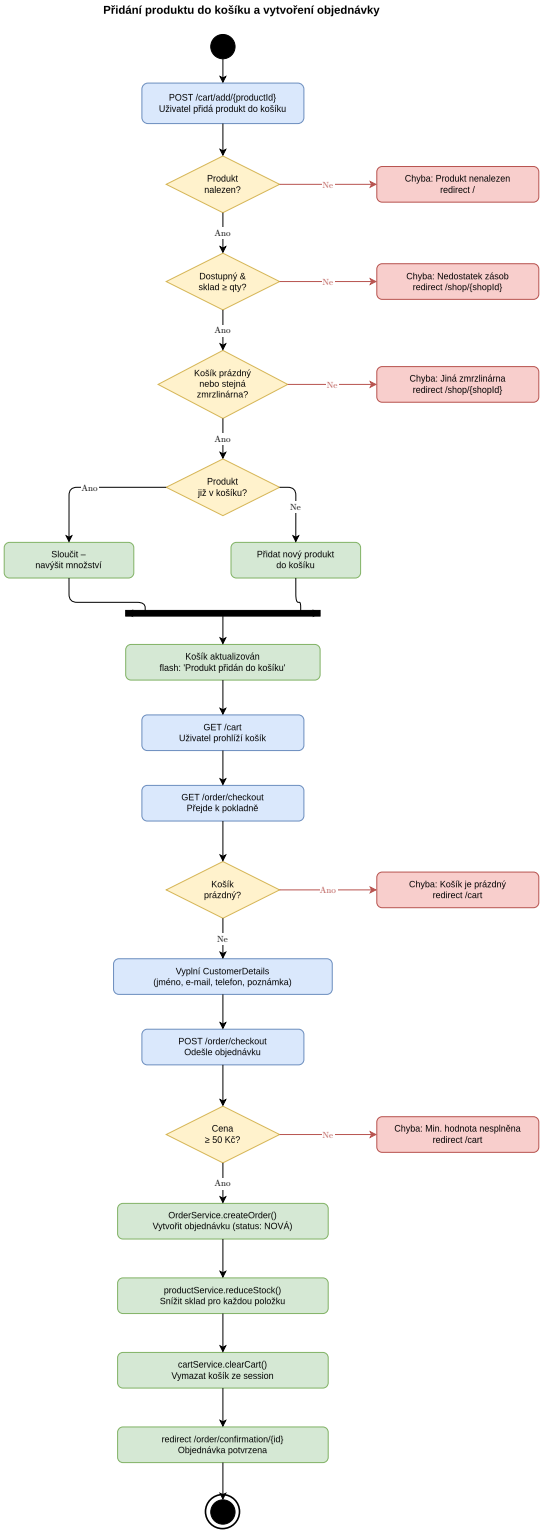
cena = (kornout/kelímek) + (počet kopečků) × 25 + (počet posypek) × 10 + (počet sušenek) × 15 (Kč)

ID	kornout/kelímek	kopečky	posypky	sušenky	cena (Kč)
PW1	`kornout`	1	0	0	$15 + 25 = \mathbf{40}$
PW2	`kornout`	3	3	2	$15 + 75 + 30 + 30 = \mathbf{150}$
PW3	`kornout`	5	6	5	$15 + 125 + 60 + 75 = \mathbf{275}$
PW4	`kelimek`	1	3	5	$10 + 25 + 30 + 75 = \mathbf{140}$
PW5	`kelimek`	3	6	0	$10 + 75 + 60 = \mathbf{145}$
PW6	`kelimek`	5	0	2	$10 + 125 + 30 = \mathbf{165}$
PW7	`kornout`	1	6	2	$15 + 25 + 60 + 30 = \mathbf{130}$
PW8	`kelimek`	3	0	5	$10 + 75 + 75 = \mathbf{160}$
PW9	`kornout`	5	3	0	$15 + 125 + 30 = \mathbf{170}$

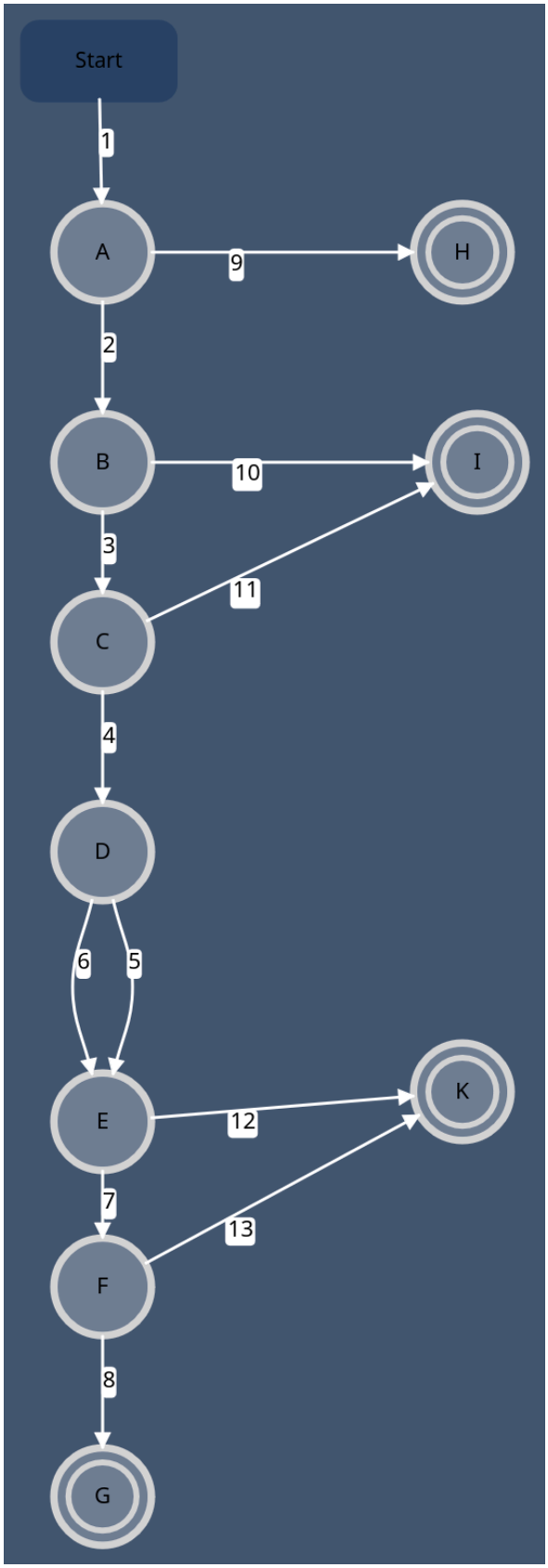
Testy průchodů

Popis procesu

Testovaný proces pokrývá **přidání produktu do košíku** (`POST /cart/add/{productId}`) a navazující **checkout** (`POST /order/checkout`). Diagram níže zachycuje průchod jako UML Activity Diagram.



CPT manager diagram



Implementace testů

Testy v rámci jedné třídy jsou na sobě závislé — spouštějí se v pořadí určeném anotací `@Order` a dřívější testy připravují data (objednávky, produkty, zmrzlinárny v databázi (pro účely testu se data mockují do **data/*.json**)), se kterými pracují testy pozdější.

IceCreamConfigurationPairwiseTests.java

Realizace **PAIRWISE** testu ze sekce Pairwise testování konfigurátoru.

Jedná se o parametrizovaný test.

Klíčové části aplikace k otestování:

- správnost kombinací jednotlivých vstupů (obal, kopečky, posypky, sušenky)
- správnost vybrání pouze validních hodnot
- správnost kalkulace ceny nakonfigurované zmrzliny
- ověření uložení správné ceny v CartItem

CSV data provider: `src/test/resources/pairwise/ice-cream-pairwise.csv`

CreateOrderProcessTests.java

Realizace **TESTU PROCESU** ze sekce Testy průchodů

- správnost průchodu všemi možnými kombinace procesu, maximální hloubky 2

Testy jsou implementovány jako Selenium end-to-end testy ve třídě `CreateOrderProcessTests` (rozšiřuje `SeleniumBaseTest`). Každý testovací případ odpovídá jedné cestě z CPT diagramu hloubky 2. Samotný testovaný flow (přidání do košíku, checkout, potvrzení) probíhá výhradně přes UI pomocí Page Object tříd `ShopDetailPage`, `CartPage`, `CheckoutPage` a `OrderConfirmationPage`. `ProductService` je využita pouze pro přípravu testovacích dat — výběr vhodného produktu podle zásob nebo ceny před začátkem testu (viz přednáška 5, způsoby přípravy dat).

NoAccountClientOrderTests.java

Testy se soustředí na všechny funkcionality dostupné pro nepřihlášeného uživatele, krom konfigurátoru zmrzliny, který je testován v IceCreamConfigurationPairwiseTests.java. Klíčové části aplikace k otestování:

- kvíz pro výběr zmrzliny
- přidání nového produktu do košíku
- nemožnost přidání produktu z jiné zmrzlinárny do košíku
- přidání více množství produktu v košíku
- odzkoušení nemožnosti objednání pokud je celková cena pod nakonfigurovanou
- objednání produktů

Test kvízu pro výběr zmrzliny je optřen **CSV** data providerem, který testuje výsledné zmrzliny z kombinací odpovědí.

CSV data provider: `src/test/resources/quiz/quiz-combinations.csv`

AuthClientOrderTests.java

Testy se soustředí funkcionality dostupné pro přihlášené uživatel.

Klíčové části aplikace k otestování:

- objednání produktů
- předvypnění údajů přihlášeného uživatele
- zobrazení provedených objednávek
- možnost objednávku zrušit

AdminPageTests.java

Testy se soustředí na admin dashboard, určený uživatelům s příznakem admin.

Klíčové části aplikace k otestování:

- změna stavů jednotlivých objednávek
- vytvoření zmrzlinárny
- přidání produktů
- naskladnění produktů

Testy jsou implementovány jako Selenium end-to-end testy ve třídě `AdminPageTests` (rozšiřuje `SeleniumBaseTest`). Přihlášení probíhá přes `loginAsAdmin()`. Objednávka potřebná pro test změny stavu je vytvořena programaticky přes `OrderService`, bez průchodu UI.